How the AI Answer Evaluator Works — In Plain Language

This is a tool that grades handwritten exam papers automatically. A teacher feeds it a scanned answer sheet, the correct answers, and the question paper. The tool reads the student's handwriting, marks every question, and hands back a report with scores and feedback — which the teacher can then check and correct.

This document explains the whole thing in everyday words. No prior technical knowledge is needed. (A short glossary at the end connects each plain-English name to the real technical term, in case you ever need to find it in the code.)

---

1. The big idea

Think of the tool as a **small office of specialists** working on an assembly line. A paper comes in one end; a finished, graded report comes out the other. Along the way it passes through a series of workers, and **each worker does exactly one job** — take the photos, clean them up, read the handwriting, match answers to the key, grade drawings, and so on. A **coordinator** hands the work from one specialist to the next in the right order.

Two simple habits make the whole thing reliable:

- **Everyone writes things down as they go.** Each paper gets its own folder, and every specialist drops its results into that folder as small note files. The next specialist reads those notes and adds its own. Nothing is kept "in someone's head" — if a step is slow or fails, the notes are still there and the office picks up where it left off. The website you use simply watches these notes and shows you the progress.
- **The machine never quietly guesses about marks.** Every score is either decided by a firm rule (like a multiple-choice answer being right or wrong) or judged by the AI against the teacher's answer key. Anything the tool is unsure about — messy handwriting, an answer that seems to belong to a different question, a possible trick — is **flagged for a human to look at**, never silently changed.

---

2. Before any grading starts: the teacher sets things up

The teacher fills in a short three-step form on the website.

Step 1 — The question paper. The tool reads the exam paper and pulls out each question, its full text, and how many marks it's worth. It reads the paper **page by page, several pages at once**, because that's fast and the marks for each question are printed right next to it.

Step 2 — The answer key (the correct answers). The tool reads the marking scheme, but it reads the **whole document in one go, not page by page**. There's a good reason: in a marking scheme the marks often sit in a column down the right-hand side, and if you chop the page into pieces you lose track of which marks belong to which answer. Reading it all at once keeps every answer paired with its marks. (To save time, the tool remembers keys it has already read, so re-uploading the same one is instant.)

Safety checks before spending any money. As soon as a file is uploaded, the tool does quick sanity checks *before* asking the expensive AI to read it:

- Is this actually a document with readable text, or just a flat photo/scan with no text in it? (A pure scan is the number-one cause of failure, so it's caught early.)
- Are marks missing from lots of questions?
- Do the answer key and the question paper agree on the total marks? If not, it warns you.

Who decides the marks? Sometimes the key and the paper disagree about points. The teacher chooses which one is the boss — the question paper or the answer key — and can even open a **guided editor** to fix individual point values by hand. Whatever the teacher decides is respected later when grading.

Where should reports be saved? The tool suggests a folder (organised by class and subject) and the teacher confirms it.

Grading can't begin until all of this is done: the paper is in, the key is in, the checks pass, the marks-authority is chosen, and the save folder is confirmed.

---

3. Grading one paper, step by step

Once setup is done, each answer sheet travels down the assembly line. Here are the workers, in order.

Step 1 — Take a picture of every page

The tool turns the uploaded PDF into a sharp image of each page (high resolution, so the handwriting stays crisp). Pages are numbered so they stay in order.

Step 2 — Clean up the pictures

Each page image is tidied so the AI can read it better: convert to grey, straighten a tilted page, fix the camera angle, boost the contrast, and enlarge it. One deliberate choice here: it keeps soft greys rather than forcing everything to pure black-and-white, because the faint difference between strokes (for example, an underscore `_` versus a dash `-` in code) matters and would be lost otherwise.

Optional check — Is every page the right way up?

Before reading, the teacher can be shown every page exactly as it was scanned and asked to rotate any that are sideways or upside down. This is entirely manual now — the tool doesn't try to guess the rotation, because guessing was often wrong. If the teacher confirms with no rotations, it's exactly the same as skipping this step. (This pause happens *before* the AI reads anything, so it costs nothing.)

Step 3 — The AI reads the handwriting

This is the heart of the tool. A powerful **AI that can look at images and read them** transcribes each page into text, question by question. Several clever things happen here:

- It reads each page together with a peek at the previous page, so an answer that spills across a page break is stitched back together correctly.
- It labels where each answer starts and ends, then assembles the fragments into one answer per question.
- For **code and mathematics**, where a single mis-read symbol matters, it re-reads those spots and, only if two readings agree, lets a careful "referee" fix a symbol — but it is never allowed to change an actual word. So a student's word is never "corrected" into something else.
- It knows the real list of question numbers for this exam (from the key and paper), so if it thinks it sees a question that doesn't exist, it treats that as a likely mis-read and flags it instead of inventing a question.
- It then does a round of **untangling**: putting a run of answers that got clumped together back into separate questions, moving a stray piece back to the question it belongs to, and recovering an answer that accidentally got glued onto its neighbour.

It also notes the student's name, roll number and date from the header, marks any spots that were too messy to read for certain, and saves a plain-text copy of everything it read.

Step 4 — Match each answer to the answer key

Now the tool lines up what the student wrote against the correct answers. Students label their answers in all sorts of ways ("Q1", "Ans 1", "1.", "Answer 1") — the tool treats all of these as **question 1** so they line up with the key no matter how they were written. It also copies the **full question text from the question paper** onto each item (the marking scheme leaves that blank on purpose), and it spots any "answer any one of these" choice questions.

Step 5 — Grade any diagrams or drawings

If the student drew something (a diagram, a graph, a labelled sketch), a separate specialist handles it. It works in **two passes**: first it lists what's in the drawing (shapes, labels, arrows) without looking at the answer, then it grades in detail while looking at the actual image again to catch anything missed. To save time, **this runs in the background at the same time as the written grading in Step 6**, and it raises a little "done" flag when it finishes so the grader knows to fold in the diagram scores.

Step 5b — Double-check the point totals

Before grading, the tool makes sure the marks add up correctly. It merges multi-part questions and "answer any one" choices so each question counts once, then **cross-checks every question's marks against the question paper**. If the marking scheme accidentally dropped some marks, it raises them to match the paper; if a whole question was missed, it adds it back; if something looks inflated, it flags it for a human. The rule of thumb: it can lift a wrong total up to the truth, but it never lowers a total that was already correct (unless the teacher declared the paper the boss).

Step 6 — Grade everything and write the report

Finally, every question is scored:

- **Multiple-choice and other objective questions** are graded by **firm rules, with no AI at all**. This is fast and can't be fooled — if a student writes fake instructions in their answer hoping to trick the grader, it simply doesn't matter here.
- **Written answers** go to the AI, which grades them against the correct answer, and the **answer key always sets the maximum marks** (the AI can never award more than the question is worth). The student's answer is clearly walled off from the grading instructions, and the AI is told to watch for and report any attempt to sneak in commands.
- To keep costs down, there's a **"quick check, then careful check"** approach: a faster AI grades first, and only the borderline answers (partial credit, low confidence, possibly off-topic, or a real answer that got a zero) are re-graded by the slower, more thorough AI.

- For "answer any two of three" style questions, it keeps the **best** of the answered parts and doesn't count the rest against the student.
- It folds in the diagram scores, and **flags anything suspicious for review** — messy handwriting, an answer that seems misplaced, a possible trick, a point-value oddity.

Then it produces the report: a nicely formatted PDF (with proper mathematics and code formatting, showing the *question* rather than echoing the answer, and diagram images you can click to enlarge) plus the interactive web version. It also records exactly how much the AI usage cost for this paper and how long each step took.

---

4. Grading a whole stack at once

You don't have to grade papers one at a time. You can upload **one big PDF containing many students' sheets**, and the tool splits it up:

- It looks at the **top of each page** and asks the AI, "Is this the start of a new student's sheet?" — spotting each student's name/details header or bubble-sheet block. That's how it finds the boundaries between students (it doesn't rely on QR codes or blank separator pages).
- It shows the teacher the proposed split — who starts where, their names and subjects — and the teacher can adjust boundaries, rename students, merge or split before approving.
- Once approved, it runs the exact same assembly line for **each student, one after another**, using the same shared answer key and question paper.
- The results appear as a **grid of student cards** — each showing the score and little badges (how many answers need a look, whether a trick was detected, how many you've reviewed). Clicking a card opens that student's full report.

---

5. The teacher checks and corrects the results

Nothing is final until a human says so. The tool keeps **two copies** of each result:

- The **original AI grades**, saved once and never touched, so there's always a truthful record of what the machine decided.
- A **working copy** that's only created the moment the teacher makes their first change. (If the teacher changes nothing, there's no working copy at all — the result is exactly as graded.)

On the report the teacher can:

- **Accept** a mark as-is.
- **Change a mark**, with an optional reason.

- **Re-grade a single question** after fixing the AI's reading of it — and if the text didn't actually change, it skips the slow re-grade entirely and just confirms.

Accept and change-mark decisions are **staged and saved together when the teacher clicks Submit** (with a warning if they try to leave with unsaved changes); re-grading a question saves right away. Every time the teacher saves, the **PDF is rebuilt** to match, and any marks the teacher overturned are recorded in a database as an audit trail. The "reviewed so far" count on the student grid always stays in sync.

---

6. The AI, and how it's kept honest and affordable

- **One doorway to the AI.** Every step that needs the AI goes through a **single shared connection** to an online AI service (by default a service called OpenRouter, though a privately-run AI server also works). Pictures are sent to the AI safely encoded, and if a request hiccups it retries once.
- **Different AI "sizes" for different jobs.** Reading handwriting, parsing the key, and grading each use a capable large model; the diagram helpers use smaller, cheaper ones. Which model does which job is just a **setting** you can change without touching the code — so you can swap in a bigger or smaller model per step. (As currently set up, the biggest model reads handwriting and grades, while smaller ones handle diagrams.)
- **Honest cost tracking.** For every paper, the tool records the **real amount the AI service billed** (not a guess) for each step, and adds it up so you know exactly what a paper cost.
- **Safety limits.** Each step has a time limit so a stuck step can't hang the whole job — it just gives up gracefully and moves on. And the tool only does so many AI requests at once, to stay within sensible limits.

---

7. Where everything is stored

Almost everything lives as ordinary files on disk, not in a database.

- **Each paper gets its own folder** containing: the page images, the cleaned-up images, the AI's reading of the answers, the matched-up answer key, the diagram results, the point-value check, the original grades, the teacher's working copy, the student's details, the cost record, timing, and progress notes.
- **A shared "current setup" area** holds the most recently uploaded answer key and question paper (already read into a tidy form), the marks-authority choice, and the save-folder choice.
- **A remembered-keys cache** so re-reading the same answer key is instant.

- **A database (a digital filing cabinet)** is used only lightly: it stores the teacher's overturned marks (the audit trail) and an old, now-unused answer bank. The tool runs fine without it.

Finished reports are written to the folder the teacher chose (organised by class and subject) and can be downloaded from the website.

---

8. Using the app (the website)

The website is a single page with a **three-step wizard** (question paper → answer key → answer sheet) that unlocks each step as the previous one is done, then shows the results.

- A **live progress checklist** shows which step a paper is on ("Reading handwriting", "Analysing diagrams", "Grading & building report").
- The **same report display** is reused for a single student and for each student in a batch.
- The **orientation check** shows each page and lets you rotate it with simple Left / Flip / Right buttons; the preview matches exactly what the AI will read.
- The **guided marks editor** presents the point-value fixes as easy cards to click through.
- **Mathematics and code** are shown properly formatted, with a plain-text fallback if the fancy formatting can't load.

Behind the scenes, long jobs run in the background and the page simply checks in for updates — so your browser never sits frozen waiting.

---

9. Running and hosting it

- **Self-contained package.** The whole app ships as a **container** — a sealed box with everything it needs — so it runs the same way on any machine. It serves the website through standard web-server software, and if given a storage disk it keeps all reports safe across restarts.
- **Sharing it over the internet.** A helper script can put the app online from a Mac through a secure **tunnel** (it supports a few different tunnel services). When it's exposed publicly it automatically turns on a **password prompt** and turns off developer debugging, so it's safe to share a link with a colleague.

---

10. How it's tested

The tool comes with a large automated test suite — **358 checks** across roughly 40 files — covering handwriting reading, page orientation, answer-untangling, point-value checking, upload safety checks, grading, multiple-choice and choice questions, the teacher-review flow, and cost tracking. These run without needing the internet or spending any money, so the behaviour can be verified safely.

---

11. A few places where the old notes are out of date

While mapping the system, a handful of **stale comments** turned up. They don't affect how it works, but they can mislead a reader:

- The little description files next to each specialist still mention an older AI ("Gemini"); the tool actually uses the newer Qwen AI now.
- The orientation step's own description still says it suggests a rotation automatically; in reality that's fully manual now.
- A comment claims "235 tests" when there are really 358.
- A couple of setting files list example values that don't match the ones actually used.
- A file named as if it holds cropped diagrams actually holds full pages.
- An old database-based answer look-up (and a few image helpers) are kept around but no longer used.

These are worth a quick cleanup so the notes match reality.

---

12. Glossary — plain word → technical term

For anyone who later needs to find these things in the code:

In this document	The real name
The coordinator / assembly line	`scripts/full_evaluator.py` (the pipeline orchestrator)
A specialist worker (one job)	a "skill" run as a separate subprocess under `skills/`
Take a picture of each page	ingestion — rasterising the PDF to PNG images
Clean up the pictures	preprocessing (OpenCV: deskew, contrast, straighten)
The AI reads the handwriting	vision OCR with the Qwen3-VL model (`run_ocr.py`)
Untangling split/mixed answers	segmentation-repair layers
Match answers to the key	ground-truth alignment

In this document	The real name
Double-check point totals	marks reconciliation against the question paper
Quick check then careful check	the grading "cascade"
A student trying to trick the grader	prompt injection
Note files in each paper's folder	JSON files under `output/<run_id>/`
The original grades vs the working copy	`review_state.json` vs `review_render.json`
One doorway to the AI	the shared `generate()` client (`scripts/llm_client.py`)
Which model does which job (a setting)	environment variables (e.g. `OCR_MODEL`, `EVAL_MODEL`)
The online AI service	OpenRouter (any OpenAI-compatible endpoint)
The digital filing cabinet	a PostgreSQL database
Sealed box that runs anywhere	a Docker container
Secure internet link	a Tailscale / Cloudflare / ngrok tunnel
The website	the Flask web app (`evaluation_app/app.py`, `index.html`)

> A companion document with the full technical detail and exact file/line references is kept in this project's version history — ask a developer if you need it.